Generative AI for Demystifying Legal Documents

Full Project Blueprint — Architecture, Pipeline, Tech Stack & Delivery Plan

A purpose-built reasoning system, not a general-purpose legal chatbot

This document is the complete build blueprint for the problem statement **"Generative AI for Demystifying Legal Documents."** It is written so that every requirement in the official Challenge and Objective — a **reliable first point of contact** for legal documents that is **private, safe, and supportive**; that gives **clear summaries**, **explains complex clauses**, **answers user queries simply**, and **helps users protect themselves from financial and legal risk** — is explicitly mapped to a concrete piece of the system. Nothing here is a generic "chatbot wrapped around an LLM." Every component exists to turn a legal document into **structured, auditable, cited data** first, and only then into language a non-lawyer can act on.

> **Core design principle:** Anyone can put Gemini behind a chat box. This system wins on trust and reliability because every output is grounded in a fixed schema, a domain-specific clause taxonomy, retrieval-based citations, and a reproducible risk score — not just fluent prose. A judge (or a real user) can trace every claim the system makes back to an exact sentence in the original document.

Problem Statement → System Feature Mapping

Requirement from brief	How this system satisfies it
Reliable first point of contact	Structured pipeline (OCR → classify → score → explain → answer) gives consistent, repeatable output on any document, not one-off generation.
Private & safe	Per-session processing, no long-term document retention beyond active use, standing disclaimer, confidence thresholds that force "consult a professional" instead of a guess.
Supportive	Plain-language explanations, color-coded risk highlights, and a scenario simulator that walks through real consequences — not just a wall of legal text.
Clear summaries	Per-clause plain-language generation required to cite its source span; readability improvement measured with Flesch–Kincaid.
Explain complex clauses	Fixed 8-category clause taxonomy with a 3-tier risk rubric — every clause is classified and explained against a defined rulebook, not ad hoc.
Answer queries simply	Grounded RAG chat: retrieves the relevant clause via vector search, answers strictly from retrieved text, shows the citation.
Protect from financial/legal risk	Risk scoring via rubric rules + embedding similarity to a labeled reference set of fair/unfair clauses — measurable, reproducible risk, not opinion.
Google Cloud GenAI (theme requirement)	Document AI, Vertex AI Gemini (structured output), Vertex AI Vector Search, Cloud Run, Firestore — all native Google Cloud services.

The rest of this document expands each row above into a build-ready specification.

1 System Architecture

The system is a four-stage pipeline. Each stage takes structured input from the one before it and produces structured output for the one after it — nothing is passed as loose, unparsed text once it leaves the ingestion stage. This is the single biggest difference between this system and a chatbot: a chatbot has one stage (“LLM reads a big blob of text, replies”); this has four, each independently testable and auditable.

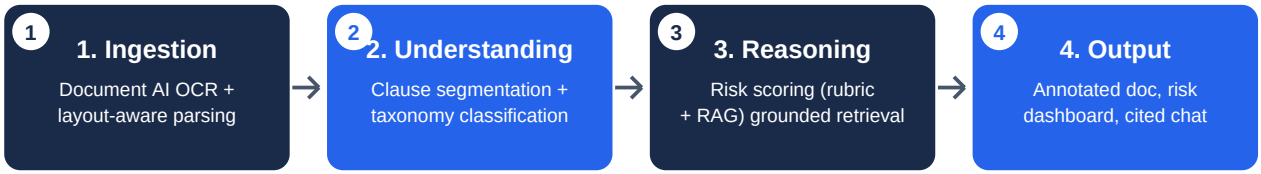

Figure 1. End-to-end pipeline — each stage emits structured JSON consumed by the next.

Stage	Responsibility	Key output
Ingestion	OCR + layout-aware parsing. Handles scanned leases, tables, and multi-column Terms of Service — not a naive PDF-to-text dump.	Raw text + layout/positional metadata per page
Understanding	Splits the document into clause-level chunks along structural boundaries (numbered sections/sub-clauses); classifies each clause into the fixed domain taxonomy.	{clause_id, category, source_text_span}
Reasoning	Scores each clause's risk using a rubric rule-check plus embedding similarity against a labeled reference set; retrieves grounded context for user Q&A.	{risk_level, risk_reason, similarity_score}
Output	Renders plain-language explanations, the annotated/highlighted document, the risk dashboard, and the cited chat interface.	Annotated PDF, dashboard JSON, chat transcript with citations

Why a pipeline instead of a single LLM call

- A single prompt (“here's a lease, summarize it”) cannot guarantee consistent categories across documents — the taxonomy step forces every clause into one of 8 fixed buckets every time.
- Risk becomes measurable: a rubric + similarity score can be evaluated for precision/recall against a labeled set; an LLM's free-text opinion cannot.
- Each stage fails independently and visibly — if classification is wrong, it shows up as a wrong category, not a vague, unreviewable paragraph.
- Structured state (Firestore records, not a chat transcript) is what makes a real UI — annotated document, dashboard, filters — possible at all.

2 Tech Stack (Google Cloud–native)

Every layer maps to a Google Cloud GenAI service, satisfying the theme requirement, while keeping room to use open-source orchestration (LangChain / LlamaIndex) if that's faster for the team — the requirement is which *models/services* power the system (Gemini, Document AI, Vertex AI), not that the team avoids all open-source tooling.

Layer	Service	Why this service, specifically
OCR / document parsing	Document AI	Layout-aware extraction — preserves tables, columns, and structure instead of flattening a scanned lease into a single unstructured string.
Generation & classification	Vertex AI — Gemini (function calling / structured output)	Forces the model into fixed-schema JSON output for clause classification and explanations, instead of free-form prose that's hard to render or audit.
Embeddings + semantic search	Vertex AI Vector Search	Powers RAG retrieval for the chat, and the “compare this clause to known-fair/unfair clauses” similarity signal that drives risk scoring.
Structured storage	Firestore (or Cloud SQL)	Stores parsed clauses, risk scores, and per-document chat history. Firestore is fastest to stand up for a hackathon timeline.
Backend / API	Cloud Run / Cloud Functions	Stateless endpoints per pipeline step (ingest, classify, score, chat) — scales independently, easy to demo and debug stage by stage.
Frontend	Streamlit (fastest) or React on Firebase Hosting	Renders the annotated PDF view, the risk dashboard, and the chat panel. Streamlit is the pragmatic choice under time pressure.
Multilingual (optional)	Cloud Translation API	Translates summaries/explanations to Hindi or other regional languages — a strong differentiator for accessibility.
Accessibility (optional)	Text-to-Speech / Speech-to-Text	Lets users hear explanations read aloud or ask questions by voice — relevant for users who struggle with dense text generally, not just legal jargon.

3 The Domain Artifact: Clause Taxonomy & Risk Rubric

This is the piece that is genuinely the team's own intellectual work, and it should be designed **before** any code is written. It is also the strongest answer to “isn't this just ChatGPT?” — the taxonomy and rubric below are what the classifier is graded against, and what gets shown to judges or users as the system's reasoning, instead of a black box.

Clause category	What to detect	High-risk trigger example
Termination / Exit	Who can end the agreement, and the required notice period.	Unilateral termination rights; no notice period; one-sided exit terms.
Penalty / Fees	Late fees, early-exit penalties.	Uncapped or disproportionate penalties relative to the harm caused.
Auto-renewal	How and when the agreement renews.	Silent auto-renewal with no reminder or opt-out window.
Liability	Who is responsible for what, and under what conditions.	Liability shifted entirely onto the weaker party (tenant/borrower/user).
Dispute Resolution	Arbitration clauses, governing jurisdiction.	Mandatory arbitration that waives the right to court entirely.
Data / Privacy (ToS)	What data is collected, and who it is shared with.	Broad third-party data-sharing rights with no meaningful limit.
Deposit / Refunds	Conditions under which a deposit is returned.	Vague or fully discretionary grounds for deposit forfeiture.
Amendment Rights	Who can change the terms after signing, and how.	Unilateral right to change terms without the other party's consent.

Three-tier risk rubric

For every category above, define Low / Medium / High risk with a one-line rule each. Example for **Termination / Exit**:

LOW — Both parties have symmetric termination rights with a reasonable, clearly stated notice period (e.g., 30 days).

MEDIUM — One party has a shorter notice period than the other, or notice terms are ambiguous but not absent.

HIGH — Only one party may terminate at will, or no notice period is specified at all.

Reference dataset (grounds the similarity-based risk score)

- Collect ~20–40 example clauses **per category**, labeled fair vs. unfair, sourced from public consumer-protection guides, sample rental/loan agreement templates, and published “unfair contract terms” lists.
- Embed the reference set with Vertex AI embeddings and store it in Vertex AI Vector Search.
- Every new clause pulled from a user's document is compared against this set — the similarity score is the quantitative half of the risk score, alongside the rubric rule-check.

4 Core Pipeline — Build Order

Build in this exact order. Every step below outputs structured JSON; that consistency is what allows the team to build a real UI on top of it, instead of parsing free text on the frontend.

1. **Upload & OCR**
   Document AI extracts text and layout metadata from the uploaded document (PDF, scan, or image).

2. **Clause segmentation**
   Split the document on structural boundaries — numbered sections and sub-clauses — rather than fixed token windows, so a clause is never cut mid-sentence.

3. **Clause classification**
   Gemini with structured/function-call output, forced into the taxonomy schema: {clause_id, category, source_text_span}.

4. **Risk scoring**
   Rubric rule-check combined with embedding similarity to the reference set → {risk_level, risk_reason, similarity_score}.

5. **Plain-language explanation**
   Generate a per-clause explanation that is required to cite its source span; if the model cannot ground the explanation, it must flag uncertainty instead of guessing.

6. **RAG Q&A**
   A user's natural-language question retrieves the relevant clause(s) via vector search; the answer is generated strictly from the retrieved text, with the citation shown alongside it.

7. **Rendering**
   Annotated PDF with color-coded risk highlights, the risk dashboard, and the chat panel are rendered from the structured data produced above.

5 Guardrails & Trust Layer

This is the explicit engineering answer to the brief's **“private, safe, and supportive”** requirement. Call this section out specifically when presenting the system — it is what separates a demo toy from something a real user could actually rely on.

Guardrail	What it does
Citation enforcement	No generated claim ships without a linked source span in the original document.
Hallucination check	A lightweight verifier step (a second Gemini call) confirms an explanation is actually supported by its cited text before it is displayed.
Confidence thresholds	Below a similarity/confidence cutoff, the system responds “we're not confident — consult a professional” instead of guessing.
Standing disclaimer	A persistent, non-buried notice: this is informational output, not legal advice.
Privacy by design	Documents are processed per-session and not retained beyond what the session needs — directly matching “private, safe” in the brief.

6 UI/UX Deliverables

Ranked by demo impact relative to build cost:

#	Deliverable	Notes
1	**Annotated document view**	Original document with clauses highlighted by risk color. Highest visual impact; cheap to build once structured output exists.
2	**Risk dashboard**	Overall trust score plus a breakdown by clause category.
3	**Grounded chat**	Q&A panel with visible citations for every answer.
4	**Readability score, before/after**	A concrete number (Flesch–Kincaid or similar) that visibly moves — easy proof point for judges.
5	Scenario simulator (stretch)	“What happens if I leave early?” — walks through actual consequences drawn from the document.
6	Multilingual toggle (stretch)	Regional-language output via Cloud Translation API.

7 Build Phases

Scale this to the actual available timeframe. Do not start a later phase before the previous one is solid — a working core beats an ambitious broken system.

Phase	Focus	Exit criteria
Phase 1	Core pipeline on one clean document	Document AI → segmentation → classification → basic risk scoring returns reliable structured JSON, before any UI work starts.
Phase 2	Grounded Q&A + annotated view	Vector search + RAG chat with citations working; highlights rendered on the document.
Phase 3	Trust layer + metrics	Hallucination check, confidence thresholds, readability scoring, and the dashboard are all live.
Phase 4	Polish + stretch features	Multilingual, voice, scenario simulator — only attempted if Phases 1–3 are already solid.

Short hackathon window (24–48h): cut straight to Phase 1 plus a minimal version of Phase 2, and use the readability/citation metrics from Phase 3 sparingly but visibly in the live demo.

8 Evaluation Metrics to Present

Numbers beat adjectives when presenting this system — each metric below is cheap to compute and directly demonstrates a claim from the brief.

Metric	What it proves
Readability improvement	Flesch–Kincaid score of the original clause vs. the plain-language output — proves “clear summaries.”
Citation grounding rate	% of generated explanations with a valid, verified source span — proves the system is not hallucinating.
Risk detection accuracy	Precision/recall against the labeled reference set; even a hand-labeled test set of 15–20 clauses is enough to produce a real number.
Latency	End-to-end time from upload to full report — proves “reliable first point of contact” in practice, not just in theory.

9 Demo Script

1. Upload a real, messy document — a scanned rental agreement with a genuinely bad clause the team knows about — not a clean synthetic one.

2. Show the annotated view; point at 2–3 real risky clauses the system caught.

3. Ask a natural-language question live ("can they keep my deposit if I leave early?") and show the grounded, cited answer.

4. Show the readability score jump and the risk dashboard.

5. Close with the differentiation line: "This isn't a chatbot summarizing text — it's a structured system that classifies, scores, and grounds every claim against reference data, so the output is consistent and auditable, not just fluent."

10 "Isn't This Just ChatGPT?" — The Answer

Have this ready, close to verbatim, for judges or stakeholders:

- Fixed schema + taxonomy → consistent, structured output across any document, not one-off prose.
- Risk scoring is comparative — embedding similarity to labeled reference clauses — not just an LLM's opinion; it's measurable and reproducible.
- Every claim is grounded with a citation and verified before display; a raw chat answer cannot guarantee that.
- Real-world messy input — scans, tables — is handled via Document AI, not just clean pasted text.
- Persistent structured state powers UI (annotated PDF, dashboard) that a chat transcript alone cannot.

> **In one line:** a chatbot answers questions about a document you feed it. This system first turns the document into verifiable, structured knowledge — classified clauses, scored risk, cited evidence — and only then lets you talk to it. The chat is the last mile, not the whole product.